

A procesat timp de 22s >

Da, situația s-a schimbat serios. **Pentru o instalare nouă nu aș mai alege MinIO Community:** repository-ul open-source AGPL a fost arhivat pe **25 aprilie 2026**, iar produsul dezvoltat activ este AIStor. Există și un nivel AIStor Free, dar este sub licența proprie MinIO, iar variantele enterprise sunt comerciale.  GitHub +2

Recomandarea mea

1. RustFS — cel mai apropiat înlocuitor pentru MinIO

Este alegerea mea pentru un cluster mic sau mediu:

- API compatibil S3;
- interfață web de administrare;
- Apache 2.0;
- rulează standalone, Docker sau Kubernetes;
- versioning, lifecycle, Object Lock și replicare;
- poate folosi clienți și SDK-uri care funcționau cu MinIO, schimbând endpoint-ul.

 GitHub +3

Exemplu de înlocuire:

```
https://minio.example.ro:9000
↓
https://rustfs.example.ro:9000
```

În multe aplicații modifici doar:

```
S3_ENDPOINT
ACCESS_KEY
SECRET_KEY
REGION
```

Atenție: RustFS a fost anunțat oficial ca Beta în aprilie 2026. L-aș instala întâi într-un mediu de test și aș valida exact aplicațiile folosite înainte de a muta date critice.  rustfs.com

2. Ceph RGW — pentru producție critică și capacitate mare

Alege Ceph dacă ai:

- cel puțin 3–4 servere dedicate;
- zeci sau sute de TB;
- nevoie de redundanță serioasă;
- administratori care pot gestiona un cluster mai complex;
- storage comun pentru S3, block și eventual filesystem.

Ceph Object Gateway oferă o implementare matură a unei părți largi din API-ul S3, iar versiunile active primesc actualizări și corecții de securitate. Dezavantajul este complexitatea semnificativ mai mare față de MinIO sau RustFS.  docs.ceph.com +1

3. SeaweedFS — bun pentru fișiere multe și infrastructură mixtă

Este Apache 2.0 și poate furniza simultan:

- object storage S3;
- filesystem;
- mount POSIX/FUSE;
- scalare pentru foarte multe fișiere.  GitHub +1

L-aș folosi pentru aplicații, documente, imagini, loguri și obiecte obișnuite. **Nu l-aș alege momentan pentru Veeam immutable storage sau cerințe stricte WORM/Object Lock**, deoarece au existat incompatibilități concrete în aceste scenarii.  GitHub +1

4. Garage — foarte simplu, dar cu funcționalitate S3 mai limitată

Garage este foarte ușor, are un singur binar și este potrivit pentru cluster mici sau distribuite între locații. Totuși, nu suportă încă object versioning, iar compatibilitatea lifecycle și replication este parțială. Nu l-aș pune pentru backupuri serioase sau aplicații care folosesc toate funcțiile S3.  Garage +1

Ce aș alege concret

Situație	Alegere
Înlocuitor simplu pentru MinIO, 1–4 servere	RustFS
Storage critic, cluster mare, multe discuri	Ceph RGW
Foarte multe fișiere și acces S3 + filesystem	SeaweedFS
Cluster mic, geo-distribuit, funcții S3 de bază	Garage

Situatie

Veeam immutable / WORM critic

Alegere

Ceph RGW sau solutie validată oficial de Veeam

Pentru mediul tău aş începe cu RustFS într-un laborator, păstrând MinIO temporar şi sincronizând bucketurile cu `rc1one` sau `mc mirror`. Pentru date critice şi mai multe noduri, aş merge direct pe Ceph RGW.



am minio nou si un bucket creat. vreau pentru acesta un access key

A procesat timp de 10s >

În MinIO-ul nou, Access Key-ul nu se leagă direct de bucket. Creezi un Access Key şi îi pui o politică ce permite acces numai la bucketul dorit. Access Key-ul moşteneşte drepturile utilizatorului părinte, iar politica inline îl poate restricţiona suplimentar. [MinIO Al...](#) +1

Varianta din interfaţa MinIO

Intră în consola MinIO şi mergi la:

Access → Access Keys → Create Access Key

Activează:

Restrict beyond user policy

Apoi introdu politica de mai jos, înlocuind NUME_BUCKET :

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket",
        "s3:GetBucketLocation",
```

```

        "s3:ListBucketMultipartUploads"
    ],
    "Resource": [
        "arn:aws:s3:::NUME_BUCKET"
    ]
},
{
    "Effect": "Allow",
    "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:AbortMultipartUpload",
        "s3:ListMultipartUploadParts"
    ],
    "Resource": [
        "arn:aws:s3:::NUME_BUCKET/*"
    ]
}
]
}

```

Apasă **Create**. MinIO îți va afișa:

```

Access Key: xxxxxxxxx
Secret Key: xxxxxxxxxxxxxxxxxxxxxx

```

Salvează imediat **Secret Key** ; parola poate să nu mai fie afișată ulterior. Consola permite generarea automată sau introducerea manuală a celor două valori și permite atașarea unei politici restrictive.  [MinIO AIStor Do...](#)

Varianta din linia de comandă

Configurezi conexiunea:

```
mc alias set minio https://minio.exemplu.ro \
MINIO_ROOT_USER \

```

Bash

Creezi fișierul:

Bash

Pui politica JSON de mai sus și apoi creezi Access Key-ul:

```
mc admin accesskey create minio/ \
```

Bash

MinIO va genera automat Access Key și Secret Key. Comanda oficială pentru versiunile noi este `mc admin accesskey create`; vechea denumire era „service account”.

 MinIO AIStor Do... +1

Poți alege și valorile manual:

```
mc admin accesskey create minio/ \
  --access-key aplicatie-bucket1 \
  --secret-key 'O_PAROLA_LUNGA_SI_RANDOM' \
```

Bash

Verificare:

```
mc alias set test-bucket https://minio.exemplu.ro \
  aplicatie-bucket1 \
  'O_PAROLA_LUNGA_SI_RANDOM'
```

```
mc ls test-bucket/MIIMF BUCKET
- .
```

Pentru acces numai la citire, elimini `PutObject`, `DeleteObject` și `AbortMultipartUpload`.



nu am Access → Access Keys → Create Access Key . Am
minio community edition

A procesat timp de 12s >

Ai dreptate. În versiunile noi MinIO Community Edition, consola web a fost redusă practic la **Object Browser**, iar administrarea utilizatorilor, politicilor și Access Key-urilor se face din `mc`. Răspunsul meu anterior era valabil pentru consola AIStor, nu pentru consola Community Edition.  GitHub +1

În Community Edition, cea mai simplă variantă este să creezi un **utilizator dedicat**. Numele utilizatorului va fi Access Key, iar parola lui va fi Secret Key.

Înlocuiește în exemple:

```
arhiva-documente    = numele bucketului
aplicatie-arhiva     = Access Key dorit
```

1. Configurează clientul `mc`

```
mc alias set local \
  http://127.0.0.1:9000 \
  MINIO_ROOT_USER \
  MINIO_ROOT_PASSWORD
Bash
```

Dacă MinIO este accesibil prin HTTPS:

```
mc alias set local \
  https://minio.domeniu.ro \
  MINIO_ROOT_USER \
  MINIO_ROOT_PASSWORD
Bash
```

Verificare:

```
Bash
```

2. Creează politica pentru bucket

```
cat > /tmp/arhiva-documente-policy.json <<'EOF'
{
```



```
SECRET_KEY="$(openssl rand -base64 36 | tr -d '\n')"
```

Bash

Salvează valoarea afișată.

5. Creează utilizatorul / Access Key-ul

```
mc admin user add \
  local \
  aplicatie-arhiva \
  "$SECRET_KEY"
```

Bash

Aici:

```
Access Key = aplicatie-arhiva
Secret Key = valoarea din $SECRET_KEY
```

Un utilizator nou nu primește implicit nicio politică și nu poate accesa nimic până când politica este atașată.  MinIO AIStor Do...

6. Atașează politica utilizatorului

```
mc admin policy attach \
  local \
  arhiva-documente-rw \
  . . . . .
```

Bash

Sintaxa corectă este:

Bash

 MinIO AIStor Do...

7. Verifică utilizatorul

Bash

Ar trebui să vezi ceva asemănător:

```
AccessKey: aplicatie-arhiva  
Status: enabled  
PolicyName: arhiva-documente-rw
```

8. Testează cheia

Configurează un alias folosind cheia nouă:

```
mc alias set test \  
http://127.0.0.1:9000 \  
aplicatie-arhiva \  
"*****"  
Back
```

Test de listare:

Bash

```
mc ls test/arhiva-documente
```

Test de încărcare:

Bash

```
echo "test MinIO" > /tmp/test.txt  
mc cp /tmp/test.txt test/arhiva-documente/
```

Test că nu vede alt bucket:

Bash

```
mc ls test/alt-bucket
```

Ar trebui să primești:

```
Access Denied
```

Comenzile esențiale, grupate

Bash

```
BUCKET="arhiva-documente"
ACCESS_KEY="aplicatie-arhiva"
SECRET_KEY="$(openssl rand -base64 36 | tr -d '\n')"
```



```
mc admin policy create local "${BUCKET}-rw" "/tmp/${BUCKET}-policy.json"
mc admin user add local "$ACCESS_KEY" "$SECRET_KEY"
mc admin policy attach local "${BUCKET}-rw" --user "$ACCESS_KEY"
```



```
echo "ACCESS_KEY=$ACCESS_KEY"
echo "SECRET_KEY=$SECRET_KEY"
```

În această variantă, `aplicatie-arhiva` este chiar Access Key-ul folosit de aplicație; nu mai trebuie să creezi separat un service account.



ChatGPT poate face greșeli. Verifică informațiile importante.